Chapter 5 Distributed Transactions

Cross service operations need atomicity rebuilt above the database boundary.

Network timeouts make it impossible to distinguish a failed commit from a lost response.

5.1 Two-Phase Commit

A coordinator asks participants to vote, then decides commit or abort.

The coordinator is a single point of failure and participants block while holding locks.

A participant that voted yes cannot decide on its own: it must wait for the coordinator decision.

Presumed abort and presumed commit recovery protocols reduce the blocking window at the cost of extra rules.

5.2 TCC

TCC splits work into Try, Confirm and Cancel business operations.
It moves the lock from database rows to business resources and needs idempotency.
Try reserves inventory or credit, Confirm consumes the reservation and Cancel releases it.
Empty rollback protection matters: Cancel may arrive before Try when the network reorders requests.
Suspension is required when a Cancel arrives while Try is still running, otherwise the resource leaks.

5.3 Saga

Saga runs local transactions and compensates them in reverse order on failure.
It does not provide isolation, so intermediate states are visible to readers.
Compensations must be idempotent because a retry can execute the same step twice.
Orchestration keeps the workflow in one service, while choreography distributes it across events.
Saga suits order fulfilment and travel booking, where intermediate states are acceptable.